Secure ATM - Final Design Document - Group 13

S. No.	Roll No.	GitHub ID	Name
1	22110047	`bp0609`	Bhavik Patel
2	22110089	`guntas-13`	Guntas Singh Saran
3	22110184	`Zeenu03`	Jinil Patel
4	23110276	`rishabhh-7`	Rishabh Jogani

1. System Overview

We build two C++17 programs - `bank` (server) and `atm` (client) - that communicate over TCP through an encrypted, authenticated channel. Security is bootstrapped from a single **pre-shared auth file** containing a 256-bit master secret, distributed out-of-band over a trusted channel before any communication begins. The bank maintains an **in-memory account ledger**; the ATM is stateless and executes one transaction per invocation.

Because both sides already share a secret, we use **symmetric cryptography only** - no TLS, no certificates, no public-key operations - the is the correct and sufficient choice when a pre-shared key exists.

---

2. How the Protocol Runs

Key Setup

On first launch, the bank generates 256 random bits (`K_MASTER`), writes them to the auth file, and prints `"created\n"`. Both sides then derive three purpose-specific keys using HKDF-SHA256:

- `K_ENC` - for AES-256-GCM encryption
- `K_MAC` - for outer HMAC-SHA256 integrity
- `K_CARD` - for deterministic card-secret derivation

Wire Format

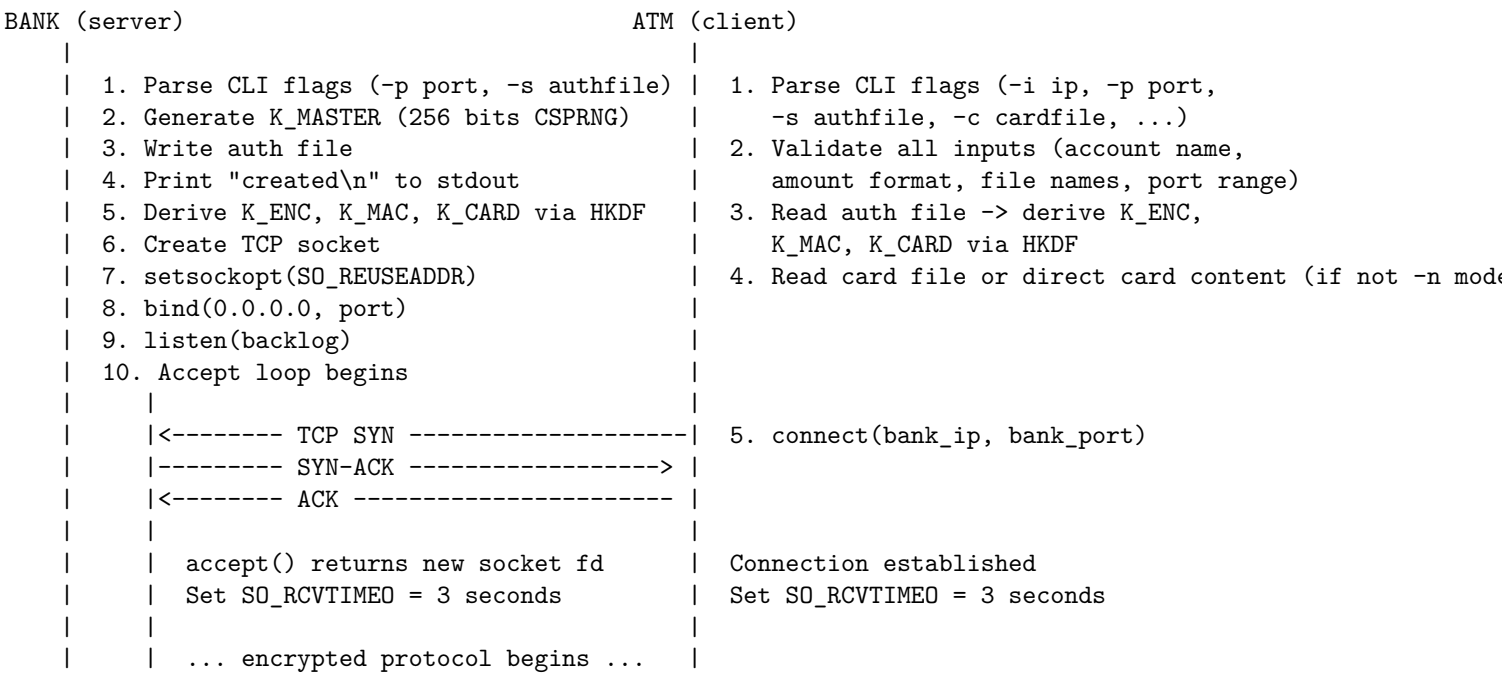
Every message is length-prefixed (4 bytes, big-endian) followed by an encrypted frame:

```
Wire Frame:
+----------+-----------+---------------------------------+------------+-------------+
| SeqNo    | Nonce     | AES-256-GCM Ciphertext + Tag    | Timestamp  | HMAC-SHA256 |
| (8 bytes)|(12 bytes) | (variable)                      | (8 bytes)  | (32 bytes)  |
+----------+-----------+---------------------------------+------------+-------------+
                                                                       |
            HMAC covers everything from SeqNo to Timestamp ------------+
```

The outer HMAC covers everything from SeqNo through Timestamp, providing a defence-in-depth authentication layer on top of GCM's built-in tag.

Initial Connection Protocol

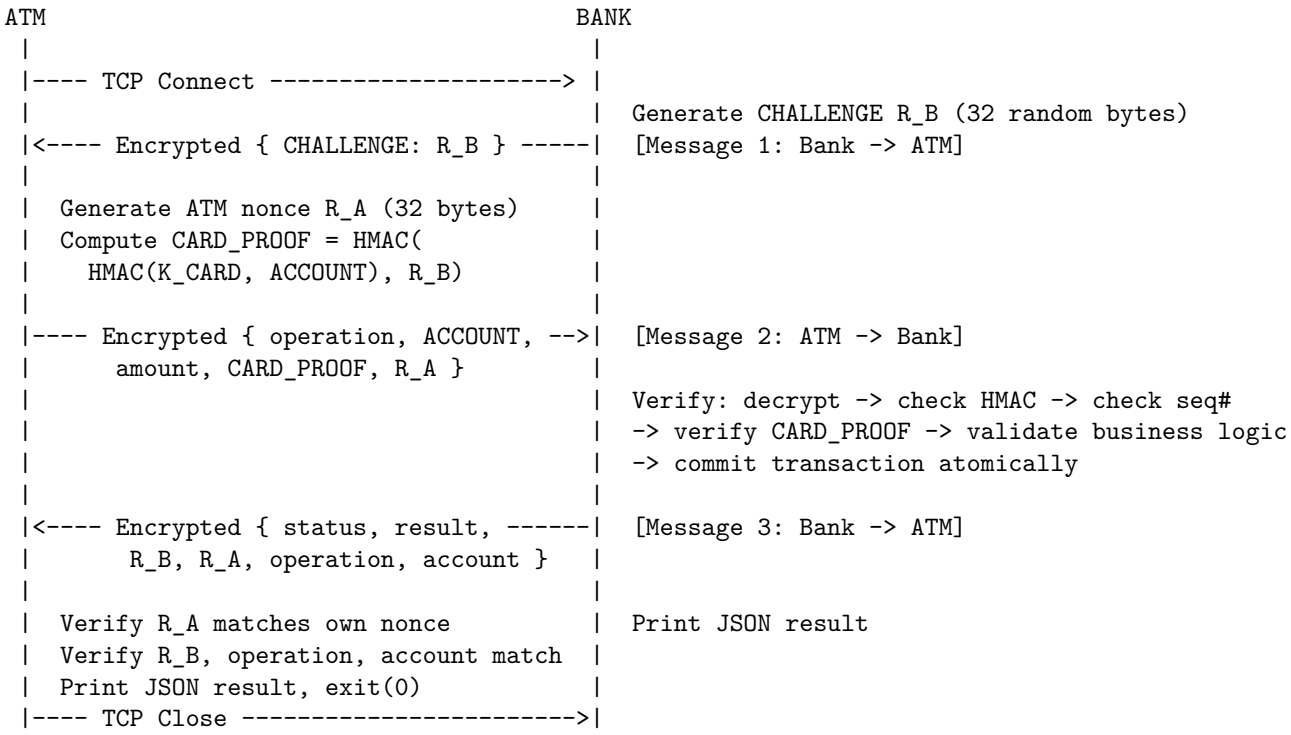
Before any encrypted messages flow, the bank must be running and the ATM must establish a TCP connection to it.



Startup ordering matters: The bank must be fully running (listening on its port and having printed `"created\n"`) before any ATM instance connects. If the ATM connects before the bank is ready, `connect()` fails and the ATM exits with code 63.

One connection per invocation: Each ATM process opens exactly one TCP connection, performs one transaction (3 encrypted messages), and exits. The bank uses a thread-per-client accept loop with a hard connection cap (`MAX_ACTIVE_CLIENTS = 60`). Every connection is subject to a 3-second socket timeout.

Transaction Flow (3 messages)



For account creation (`-n`), the flow is identical except `CARD_PROOF` is omitted - the ATM computes and writes the card secret locally after receiving the success response.

What Happens When Things Fail

Every failure has a defined, deterministic outcome. The guiding rules:

1. **Bank never crashes.** Any bad input -> print `"protocol_error\n"`, close connection, resume accepting.
2. **ATM exits with a code.** Network/protocol failures -> `exit(63)`. Validation failures or bank-reported `"fail"` -> `exit(255)`. Neither side prints anything to stdout on failure.
3. **No partial state.** The bank commits ledger changes only after all validation passes. If anything fails mid-protocol, the ledger is untouched.
4. **No information leakage.** The bank gives the same generic response regardless of whether a failure was caused by a wrong card, missing account, or insufficient funds.

Specific failure points:

- **TCP connect fails** -> ATM exits 63; bank is unaffected.
- **Decryption/HMAC/sequence/timestamp check fails** (any message, either direction) -> receiving side treats it as protocol error. Bank prints `"protocol_error\n"` and closes. ATM exits 63.
- **Card proof mismatch** -> Bank closes immediately (no response sent), prints `"protocol_error\n"`. ATM times out, exits 63.

- **Business logic failure** (account exists, insufficient funds, etc.) -> Bank sends encrypted `{status: "fail"}`, prints `"protocol_error\n"`. ATM exits `255`.
- **Response lost in transit** (bank committed but ATM never received) -> Bank's ledger is the source of truth; ATM exits `63`. A subsequent balance check reveals the real state. This is a fundamental limitation without two-phase commit, and the spec accepts it.
- **3-second timeout** applies to every connection. Prevents slowloris, hanging connections, and resource exhaustion.
- **Message size cap** (~8 KB). Any length prefix exceeding this -> immediate close without allocating memory.

---

3. Security Model

Attacker Capabilities

The man-in-the-middle can observe all traffic, modify bytes in transit, inject new messages, replay old messages, reorder messages, and delay or drop messages. They have access to the source code but **not** the auth file. They may or may not have a card file.

How We Defeat Each Attack

Attack	Defence
Eavesdropping	AES-256-GCM encrypts all content with K_ENC. Without K_MASTER, nothing is readable.
Tampering	GCM's 128-bit auth tag + outer HMAC-SHA256 detect any modification.
Replay	Fresh 32-byte random `CHALLENGE` per session + monotonic sequence numbers + timestamp window (±30s). ATM sends its own nonce `R_A` inside the encrypted request; bank echoes it in the response. Since the MITM cannot decrypt the request, it cannot replay a response with the correct `R_A`.
Reorder	Strict sequence number check: expected = previous + 1.
Injection/Forgery	Requires `K_ENC` and `K_MAC` (from auth file) to produce valid frames. Impossible without the secret.
Impersonation	Both sides implicitly authenticate through shared key derivation. A fake bank cannot produce a valid `CHALLENGE`; a fake ATM cannot produce a valid card proof.
Cross-account access	Card secret = `HMAC(K_CARD, ACCOUNT_NAME). Bob's card mathematically cannot authenticate as Alice`.
Card forgery	Requires `K_CARD`, which requires the auth file.
Timing attacks	All MAC comparisons use `CRYPTO_memcmp()` (constant-time).
DoS (connection flood)	3-second hard timeout per connection, thread-per-client handling, and a hard cap of 60 active connections. Garbage data fails fast at GCM verification (~microseconds). Bank never crashes.
Memory exhaustion	Max message size enforced before buffer allocation.

Why Symmetric-Only Works

The auth file eliminates the key-distribution problem. Public-key crypto (RSA, ECDH, TLS) solves a problem we don't have. AES-256 with a 256-bit pre-shared key provides 256-bit security - equivalent to a 15,360-bit RSA key - with a simpler implementation and smaller attack surface.

4. The Pre-Shared Symmetric Key

The auth file contains 256 bits of CSPRNG output (the master secret `K_MASTER`). We never use it directly for any operation. Instead, HKDF-SHA256 derives three domain-separated keys:

```
K_MASTER -- HKDF --> K_ENC   (encryption)
                  > K_MAC   (message authentication)
                  > K_CARD  (card secret derivation)
```

This prevents related-key attacks - even though all keys come from the same source, they are cryptographically independent. If we used K_MASTER directly for both encryption and MAC, a theoretical weakness in one primitive could compromise the other. HKDF makes this impossible.

Card File - Creation and Authentication

The card secret is `HMAC(K_CARD, ACCOUNT_NAME)` - deterministic, not random. The bank never stores card secrets; it recomputes them on the fly from `K_CARD` + `ACCOUNT_NAME`.

Creation (`-n` mode):

```
ATM                                          BANK
 |                                            |
 |  (no card file exists yet)                 |
 |---- Encrypted { create, "bob", 1000 } --> |
 |                                            |  Verify account doesn't exist
 |                                            |  Create account with initial balance
 |<--- Encrypted { success, balance } ------ |
 |                                            |
 |  CARD_SECRET = HMAC(K_CARD, "bob")         |
 |  Write CARD_SECRET to bob.card             |
 |  (or print to stdout with \texttt{-m})      |
 |  (card file never sent to bank)            |
```

The card secret is computed entirely on the ATM side after a successful create response. It is never transmitted. If the card file already exists, the ATM exits 255 before connecting.

Authentication (`-d`, `-w`, `-g` modes) - Challenge-Response:

```
ATM                                          BANK
 |  Read CARD_SECRET from bob.card           |
 |  (or pass card content with \texttt{-C})  |
 |                                           |
 |<--- Encrypted { CHALLENGE: R_B } -------- |  R_B = 32 random bytes (CSPRNG)
 |                                           |
 |  Generate R_A (32 random bytes)           |
 |  CARD_PROOF = HMAC(CARD_SECRET, R_B)      |
 |                                           |
 |---- Encrypted { op, "bob", amount, -----> |
 |      CARD_PROOF, R_A }                    |
 |                                           |  Bank independently computes:
 |                                           |    EXPECTED_SECRET = HMAC(K_CARD, "bob")
 |                                           |    EXPECTED_PROOF  = HMAC(EXPECTED_SECRET, R_B)
 |                                           |  Verify: CARD_PROOF == EXPECTED_PROOF
 |                                           |    (constant-time via CRYPTO_memcmp)
 |                                           |  Match -> proceed; mismatch -> protocol_error
 |                                           |
 |<--- Encrypted { status, result, --------- |  Echo R_A, R_B, operation, account
 |      R_B, R_A, operation, account }       |  in response
 |                                           |
 |  Verify R_A matches own nonce             |
 |  Verify R_B, operation, account match     |
```

Why this works:

- **The card secret never crosses the wire** - only `CARD_PROOF = HMAC(CARD_SECRET, CHALLENGE)` is transmitted, and it's a one-time value bound to a fresh random challenge.
- **Uniqueness is mathematical** - HMAC-SHA256 is a PRF; different account names yield different secrets with 2^{128} collision resistance.
- **Account binding is intrinsic** - Bob's card yields `HMAC(K_CARD, "bob")` != `HMAC(K_CARD, "alice")`, so Bob's card provably cannot authenticate for Alice's account.
- **Replay is impossible** - each session uses a fresh 32-byte `R_B`, so a captured `CARD_PROOF` from a previous session is useless.

---

5. Design Decisions

AES-256-GCM over AES-CBC + HMAC: Both are provably secure, but GCM combines encryption and authentication in one pass, reducing implementation bugs. It also natively supports Associated Authenticated Data (AAD), which we use to bind sequence numbers and timestamps to ciphertexts.

Outer HMAC (defence-in-depth): Strictly redundant given GCM's auth tag. We keep it as a safety net against hypothetical GCM implementation bugs and to ensure the unencrypted header fields are always covered. It costs ~200ns per message - negligible.

Sequence numbers + timestamps: For a 3-message protocol, sequence numbers alone would suffice. We add timestamps (+-30s window) as cheap insurance against cross-session replay if sequence state is somehow leaked. If we had to simplify, timestamps would be the first thing we'd drop.

Challenge-response over sending the card secret directly: Sending the secret would expose it in the encrypted payload - a problem if the encryption is ever compromised. The `CHALLENGE`-response ensures the card secret never leaves the local machine.

Cents as `uint64` internally: Avoids all floating-point precision issues. We only convert to `double` at JSON output time. The uint64 range (up to ~1.8 x 10^19) is more than sufficient.

C++ with OpenSSL: OpenSSL is pre-installed on the target system, requires no internet to build, and provides battle-tested implementations (AES-NI acceleration, FIPS validation). We use the EVP API throughout: `EVP_aes_256_gcm`, `HMAC()`, `EVP_PKEY_HKDF`, `RAND_bytes()`, `CRYPTO_memcmp()`, and `OPENSSL_cleanse()`.

---

6. Work Distribution

Guntas Singh Saran (22110089) - Crypto & Key Management

- Auth file generation (CSPRNG) and reading/parsing
- HKDF key derivation (`K_ENC`, `K_MAC`, `K_CARD`)
- AES-256-GCM encrypt/decrypt wrappers (EVP API)
- HMAC-SHA256 wrappers (card secret derivation, card proof, outer MAC)
- RAII `SecureBuffer` class with `OPENSSL_cleanse` in destructor
- Constant-time comparison via `CRYPTO_memcmp`

Files: `crypto/keys.{hpp,cpp}`, `crypto/aes_gcm.{hpp,cpp}`, `crypto/hmac.{hpp,cpp}`, `crypto/random.{hpp,cpp}`

Jinil Patel (22110184) - Protocol & Networking

- TCP socket layer: server (bind/listen/accept) and client (connect)
- Length-prefixed framing: 4-byte big-endian read/write
- Encrypted session: send = serialize -> encrypt -> HMAC -> frame; recv = deframe -> verify HMAC -> decrypt -> deserialize
- Sequence number tracking (per-direction monotonic counter)
- Timestamp generation and 30s validation
- 3-second timeout management via `setsockopt(SO_RCVTIMEO)`
- Message size cap enforcement
- SIGTERM signal handler (bank graceful shutdown)

Files: `protocol/framing.{hpp,cpp}`, `protocol/session.{hpp,cpp}`

Bhavik Patel (22110047) - ATM Client (Business Logic) & Bank Server + UI Framework (Business Logic)

- Bank `main.cpp`: CLI parsing, startup sequence, auth file creation
- Account ledger: `std::unordered_map<std::string, uint64_t>` protected by `std::mutex`
- Thread-per-client handling with connection cap (`MAX_ACTIVE_CLIENTS = 60`)

- Per-connection handler: decrypt request -> verify CARD_PROOF -> dispatch operation
- Operations: create account (check doesn't exist, balance >= 10.00), deposit (> 0), withdraw (sufficient funds), get-balance
- Atomic commit: ledger modified only after all validations pass
- JSON output to stdout (bank side), flush after every line
- All error paths -> `"protocol_error\n"`, close connection, continue serving
- ATM `main.cpp`: CLI parsing (POSIX/getopt), mode dispatch (-n/-d/-w/-g, `-m`, `-C`, `-help`)
- Card file I/O: create (write HMAC secret or `-m` to stdout), read (parse secret or `-C` direct content)
- Transaction execution: connect -> receive `CHALLENGE` -> compute CARD_PROOF -> send request -> receive response -> print JSON
- Laid the groundwork for the UI integration (see `ui/backend/`) - a local Node.js/Express server that spawns `atm` processes on demand, never exposing secrets to the browser.

Files: `build/src/atm/main.cpp`,`build/src/bank/main.cpp`, `build/include/common/exitcodes.hpp`, `build/include/common`, `ui/backend/`, `ui/frontend/`

UI Frontend & Local Backend (Web UI)

- React + Vite SPA in `ui/frontend` (hosted publicly)
- Node.js/Express backend in `ui/backend` (runs locally on each user machine)
- Backend spawns `atm` with argv, never exposing auth or card secrets to the browser
- Deployment and runtime model documented in `ui/README.md` and `ui/DEPLOYMENT.md`

```
Browser (Public SPA)
      |
      | HTTPS (static assets)
      v
Public Frontend Host (GitHub Pages)
      |
      | fetch http://127.0.0.1:4000/api/*
      v
Local Node.js Backend (user machine)
      |
      | spawn(atm, argv)
      v
Local atm binary + local bank.auth + card files
      |
      | TCP
      v
Remote bank server (public IP:port)
```

Files: `ui/frontend/*`, `ui/backend/*`, `ui/README.md`, `ui/DEPLOYMENT.md`, `.github/workflows/ui-gh-pages.yml`

Rishabh Jogani (23110276) - Input Validation

- Full input validation: account name regex `[_\-\.0-9a-z]{1,122}`, amount format `(0|[1-9][0-9]*)\.[0-9]{2}`, file names `[_\-\.0-9a-z]{1,127}` (not `./..`), IPv4 dotted-decimal, port 1024–65535, duplicate flag detection
- JSON output formatting: cents -> correct decimal representation, proper precision

- CMakeLists.txt, wrapper Makefile, build system integration
- Exit code logic: 0 (success), 63 (protocol/network error), 255 (validation/business error)

Files: `build/src/validator/validator.cpp`, `build/include/validator/validator.hpp`, `build/CMakeLists.txt`, `build/Makefile`

Integration Points

Members A and B produce the shared `common` static library (crypto + protocol + validators). Members C and D build the `bank` and `atm` executables that link against it. The interface boundary is clean: Member C and D call `session.send()` / `session.recv()` and get back decrypted, verified JSON - they never touch raw crypto.

---

7. Build System

We use CMake with a thin Makefile wrapper so that `cd build && make` works as the spec requires:

```
secure-atm/
|-- build/
|   |-- Makefile            # Build wrapper (run `make` here)
|   |-- CMakeLists.txt      # CMake build configuration
|   |-- README.md           # Build/run usage for CLI
|   |-- include/
|   |   |-- common/{types,exitcodes}.hpp
|   |   |-- crypto/{random,keys,aes_gcm,hmac}.hpp
|   |   |-- protocol/{framing,session}.hpp
|   |   |-- validator/validator.hpp
|   |   |-- nlohmann/json.hpp
|   |-- src/
|   |   |-- crypto/{random,keys,aes_gcm,hmac}.cpp
|   |   |-- protocol/{framing,session}.cpp
|   |   |-- validator/validator.cpp
|   |   |-- bank/main.cpp
|   |   |-- atm/main.cpp
|   |-- tests/
|       |-- test_framework.hpp
|       |-- test_validator.cpp
|       |-- test_crypto.cpp
|       |-- test_protocol.cpp
|       |-- test_integration.sh
|       |-- dos_attack.cpp
|-- ui/
    |-- frontend/           # React + Vite SPA (public hosting)
    |-- backend/            # Local Node.js backend (spawns atm)
```

Dependencies: C++17 compiler (g++), OpenSSL (`libssl-dev`), pthreads - all available offline on the target system. No internet required.

8. Summary

The design uses symmetric-only crypto anchored on a pre-shared 256-bit secret. HKDF derives domain-separated keys. AES-256-GCM provides confidentiality and integrity. An HMAC-based challenge-response authenticates card possession without sending secrets over the wire. Sequence numbers, timestamps, a fresh random `CHALLENGE` per session, and an ATM-generated nonce `R_A` echoed in the response prevent replay and reordering. The bank never crashes, commits atomically, and times out all connections at 3 seconds. A web UI is provided via a public frontend with a local backend that spawns `atm`. The whole thing compiles with `make` from source on an offline machine.